

Optimizing Specification-Driven Delivery

Atomic decomposition, build graphs, and context engineering for reproducible LLM delivery

Ed Barlow Web Cloud Studio July 2026



Optimizing Specification-Driven Delivery

Ed Barlow — Web Cloud Studio

Abstract

Specification-driven development makes intent durable but leaves delivery unmanaged. A specification large enough to define a real product is too large to build in one prompt: build quality degrades with prompt size. Drydock [1] closes the gap with process. Agile decomposition reduces the specification to atomic stories. The stories form the Manifest, a plain-text graph database with explicit dependencies. The build engine walks the graph and assembles, for each step, only the context that step declares: full specifications for what the step builds, compacted derivatives for what it consumes, shared enterprise rules injected once. Content hashes over applied specifications confine every rebuild to the subgraph a change touches. The result: context is engineered, not hoped for, and the application rebuilds on demand — wholly or in optimized chunks.

Keywords: specification-driven development, LLM code generation, context engineering, dependency graphs, Agile decomposition, prompt compaction, reproducible builds

1. The Problem

LLM build failures are context failures, not capability failures. A model given a bounded task with exactly the required information performs reliably. The same model given a full product specification and an open-ended instruction drifts: it drops constraints stated early in the context and re-implements components it cannot see [5].

Specification-driven development solves half the problem. The specification is the source of truth; the software improves by editing it, not by accreting conversational patches. Spec Kit [4] established the specification artifact as the unit of software intent.

The other half remains unsolved: there is no process between the specification and the build. Past a size threshold, adding specification text to a prompt reduces build fidelity. Working software that cannot be reproduced from its specification is not working software.

Drydock's answer is the two processes the industry already validated: Agile decomposition and test-driven development. A companion paper [2] covers correctness. This paper covers optimization — five mechanisms:

Mechanism	Effect
Atomic decomposition	Every unit of work fits a declared token budget
The Manifest	Work is a graph with state, dependencies, and provenance
Context stacking	Every prompt is computed from declared files, never improvised
Compaction	Implementation detail is injected once — where it is built

Mechanism	Effect
Hash-verified rebuilds	Change invalidates only the dependent subgraph

2. Atomic Units of Work

`drydock analyze` treats the imported specification as an Agile Epic and decomposes it into features and stories [1, §"SAIL Phase 2"]. Each story declares its contract:

Field	Contents
<code>implements:</code>	Specification files this story realizes in code
<code>context:</code>	Read-only supporting specifications
<code>depends:</code>	Stories and spikes that must verify before this story runs
<code>instructions:</code>	The bounded task statement

Every story names its inputs. The context of a build step is therefore a computed property of the graph, not a judgment call at prompt time. A story is atomic when its declared context fits the configured budget (`PROMPT_WARN_TOKENS` , default 50,000 tokens). Decomposition continues until every story qualifies.

Questions the specification cannot answer become **spikes**. A spike's product is a `finding:` — a recorded answer available as context to every downstream story. Research runs once and is never re-derived inside a build prompt.

Decomposition follows the system shape: web applications by route and screen, CLI tools by command, libraries by public API symbol [1, §"Specification Decomposition Methodology"]. Each specification file declares `Provides` and `Consumes` ; the planner computes `Depends On` from them. The criterion is Parnas's [6]: partition on interface boundaries.

3. The Manifest: A Graph Database of the Build

`drydock plan` writes `MANIFEST.md` , a graph build plan in structured Markdown [1, §"The Manifest"]. Nodes are `feature` , `story` , and `spike` blocks. Edges are `depends:` references between stable identifiers. Node attributes carry state, evidence paths, and prompt-assembly fields.

The graph yields three properties:

The frontier is computable. The set of blocks whose external dependencies are all `closed/verified` is readable by inspection. `drydock build` executes the frontier; passing acceptance unlocks the next set. Build order is a property of the data.

Estimation uses the native currency. The planner estimates story points in tokens — the actual unit of LLM cost. The Commander sees what each story costs before execution.

Provenance is durable. The `applied_specs` registry records, per applied specification file, its content hash, commit, applying story, and timestamp. Replanning never regenerates this registry. It is the memory of what was built from which exact text, and the foundation of incremental rebuild (§6).

The Manifest is an execution view, not a second product definition. Intent lives in the typed specification files.

4. Order of Operations and Context Grouping

Given the graph, delivery is an optimization problem: choose an order and grouping that respects dependencies, keeps every prompt under budget, and minimizes total injected context.

The token arithmetic is decisive. If k stories share c tokens of common context plus private specifications s_i , ungrouped execution injects $k \cdot c + \sum s_i$; grouped execution injects $c + \sum s_i$. In enterprise builds, c is dominated by material that never changes between stories — architecture, database contract, stack rules, branding. Grouping removes most duplicate context from a build.

The planner emits a Foundation → Data → Features → UI order and groups similar stories into build blocks. `BUILD_COMPASS.md` exposes both to the Commander, who reorders and regroups or accepts the default [1, §"Agile Build Planning"]. The ordering heuristic

Follows from the same arithmetic: build what everything consumes first, so later steps receive compact derivatives (§5) instead of full text.

Each prompt stacks a fixed file set: `COMPASS.md`, the block's `implements:` files, compact derivatives of its `context:`, the applicable stack rules, and the instructions. Nothing else. `drydock build --dry-run` prints the file list, prompt size, and token estimate without executing. Context composition is itself a reviewable artifact.

5. Compaction: Builders and Consumers

The largest source of duplicate context is implementation detail injected into steps that only use an interface. Drydock separates the two roles: the story that builds a feature receives its full specification; every story that calls it receives only the callable surface [1, §"Compaction"].

`drydock rigging compact` produces `_compact.md` derivatives: routes, signatures, typed parameters, one-line summaries. Rationale and examples are discarded. The canonical case is `DATABASE.md`: the full file specifies schemas, migrations, and a typed access-class library; `DATABASE_compact.md` carries only class names and method signatures. One foundation story sees the full file. Every feature above it sees an interface a fraction of the size.

Two rules make compaction safe at scale:

- **Compact stability.** Recomposition reproduces the existing derivative verbatim unless the extracted contract changed. An unchanged derivative keeps its bytes and hash and triggers no rebuild cascade. Cosmetic edits do not invalidate the portfolio.
- **Freshness gating.** A step requiring an absent or stale derivative stops with a directive. It never silently substitutes the full file. The context budget is a contract.

Compaction extends to the enterprise layer. **Rigging** — Drydock's tree of business rules, per-technology stack files, and branding — ships with pre-built compact derivatives. An organization writes its stack guidance once; every project's build steps receive the compacted rules for their technology. Portfolio governance stops being a per-prompt copy-paste tax.

6. Rebuilding in Optimized Chunks

Reproducibility is a mechanical consequence of provenance. Before executing any agent, `drydock build` compares every entry in `applied_specs` against current file content. A hash mismatch blocks the build and names the stale file [1, §"drydock build"].

Change localizes along graph edges:

- Editing one `FEATURE-*.md` resets to `pending` exactly the blocks whose `implements:` include it. Clean blocks keep their state. The next build rebuilds the affected subgraph and nothing else.
- Sealed foundational files (`ARCHITECTURE.md`, `DATABASE.md`, `UI-GENERAL.md`) require an explicit change ticket and `drydock refit` — matching the blast radius of foundational change.
- The Commander dirties a file deliberately to force re-application of its story.

Full rebuild is the degenerate case: empty target, frontier at the foundation, graph replayed in order. No prompt depends on conversation history; every prompt assembles from versioned files. The specification, the graph, and the hashes are the build system. `make` gave compilation declared inputs, computed order, and rebuilds proportional to change. LLM delivery deserves the same discipline.

7. Related Work

Spec Kit [4] established specification-plus-task-list as the agent interface; a Drydock story is an enriched Spec Kit task with states, dependencies, acceptance, and prompt-assembly fields, and Drydock imports Spec Kit projects directly. Agent frameworks orchestrate multi-step builds but treat context as emergent from agent memory rather than computed from a declared graph. Context degradation with prompt length is documented empirically [5]. The correctness half of the argument — acceptance declared before build, verified deterministically after — is the companion paper [2].

8. Conclusion

Decompose the Epic into token-bounded atomic stories. Record them in a plain-text graph with explicit dependencies and provenance. Compute every prompt from declared files. Compact what is consumed; inject full detail only where it is built. Let content hashes confine every rebuild to the subgraph a change touches. Software built this way rebuilds, chunk by optimized chunk, for as long as its specification lives.

References

- [1] E. Barlow. *Drydock Specification: Agile Specification-Driven Design — The SAIL Methodology for Governed Software Delivery*. Web Cloud Studio, 2026. <https://github.com/webcloudstudio/Drydock>
- [2] E. Barlow. *Improving Step Accuracy in Specification-Driven Development*. Web Cloud Studio, 2026.
- [3] K. Beck et al. *Manifesto for Agile Software Development*. 2001. <https://agilemanifesto.org>
- [4] GitHub. *Spec Kit: Toolkit for Spec-Driven Development*. 2025. <https://github.com/github/spec-kit>
- [5] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. "Lost in the Middle: How Language Models Use Long Contexts." *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- [6] D. L. Parnas. "On the Criteria To Be Used in Decomposing Systems into Modules." *Communications of the ACM*, 15(12):1053–1058, 1972.